

2.5 Example: Solving Sudoku

In this final example, we'll show you how you can develop a Java program for solving sudoku games in conjunction with ChatGPT or another AI tool. The initial situation is that you're pursuing your own ideas, but you want to avoid tedious typing with the help of AI support. So, you give clear instructions as to which code the AI should generate, then incorporate this into your program, and finally develop it further yourself until you encounter problems. If you will, you're the "senior developer" who can delegate unpleasant work to a "junior AI developer".

2.5.1 Class Setup

Working step by step, first, you specify the structure of the program. You leave the AI bot with little choice. The prompt contains information about your knowledge, the preferred programming language, the planned program structure, and the data structure for storing the sudoku game.

Single-Prompt Sudoku Solver

At the end of the section, we'll show you that you can also get a sudoku solver with a single prompt, but that isn't your goal. As the saying goes, "It's the journey, not the destination," and in the course of this journey, you want to deepen your code structuring skills.

You explicitly ask the AI chat system to set up a `Sudoku` class with instance variables and a constructor, but without any other methods. In addition, the AI tool should implement the `Main` class and initialize a simple sudoku puzzle there as you need a sudoku puzzle as a starting point for the other tests.

Prompt: I am an intermediate Java programmer. I want to write a program to solve sudokus. I want to do this step-by-step with your assistance. Please

do NOT provide the complete code. Instead, help me to set up classes, add methods, and so on in small steps.

My plan is to use (at least) two classes: Main with testing code and Sudoku to store and solve the sudoku. More classes might follow.

The sudoku is to be stored in a 9×9 int array. 0 means this cell is empty.

If this sounds OK, please provide the Sudoku class with a private instance variable for the array and a constructor to initialize the variable. The constructor should also verify the array size and its contents (only values between 0 and 9 allowed). Add no other code to the class.

Also provide the Main class, which creates a Sudoku object and passes a simple but incomplete sudoku to solve.

You copy the two classes into a new Java project and have a functional basic framework for your code in two minutes. The code for the Sudoku class looks as follows:

```
import ...;
public class Sudoku {
    private int[][] grid; // store Sudoku cells in
    array

    // constructor
    public Sudoku(int[][] grid) {
        // Check if the input grid is valid
        if (grid.length != 9 || grid[0].length != 9) {
            throw new IllegalArgumentException(
                "Grid must be a 9x9 array.");
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                if (grid[i][j] < 0 || grid[i][j] > 9)
            {
                throw new
                IllegalArgumentException(
                    "Grid values must be between 0
```

```

        and 9.");
    }
}

// Initialize the grid
this.grid = new int[9][9];
for (int i = 0; i < 9; i++) {
    for (int j = 0; j < 9; j++) {
        this.grid[i][j] = grid[i][j];
    }
}
}
}

```

2.5.2 Detaching Individual Fields in Rows

Now you can proceed as you would to solve a sudoku: in a first method, your program should run through all rows. If there is only one free field in a row, this can be filled with the only number that doesn't yet appear in the row.

In the prompt, you can formulate a loop over all rows as the first intermediate goal, but for the time being, only ask for suggestions for a meaningful name for the method.

Prompt: Perfect. Now we start the solving code. As a first step, I want to loop through the grid to find rows with exactly *one* empty cell. It should be easy to insert the missing value into these positions. I am unsure how to name this method. Do you have any ideas?

Needless to say, ChatGPT does have some ideas for the name; it provides four to choose from. The AI tool also provides an unsolicited code suggestion using the method name, `fillSingleEmptyInRow`.

You look at the code in the chat system and are basically satisfied. Before you transfer the code to your program, however, you still have two requests for improvement, which you express in the following two prompts. First, more meaningful variables should be used in the code (ChatGPT ini-

tially opted for `i` and `j`), and second, the method should integrate a counter and at the end return how many fields were solved when running through the loop.

To prevent the AI tool from outputting the entire code of the class at each iteration, ask it to display only the relevant method. Notice how the AI tool understands from the context that the two prompts refer to the previous question.

Prompt: Please replace `i` and `j` with `row` and `column` to make the code more clear.

Prompt: Add a counter to `fillSingleEmptyInRow`. The method should return the number of cells it has filled. Only output the changed method, not the entire class.

The method programmed entirely by the AI tool now reads as follows:

```
public int fillSingleEmptyInRow() {
    int cellsFilled = 0;
    for (int row = 0; row < 9; row++) {
        int emptyCount = 0;
        int emptyColumn = -1;

        for (int column = 0; column < 9; column++) {
            if (grid[row][column] == 0) {
                emptyCount++;
                emptyColumn = column;
            }
        }
        if (emptyCount == 1) {
            boolean[] present = new boolean[10];
            for (int column = 0; column < 9; column++)
            {
                present[grid[row][column]] = true;
            }
            for (int number = 1; number <= 9;
number++) {
                if (!present[number]) {
                    grid[row][emptyColumn] = number;
                    cellsFilled++;
                }
            }
        }
    }
    return cellsFilled;
}
```

```

        break;
    }
}
}
return cellsFilled;
}

```

[explanation of changes]

2.5.3 Solving Individual Fields in Columns and in Blocks

Now you incorporate the method suggested by the AI tool into your program and test it. Once everything works to your satisfaction, you ask ChatGPT to write two equivalent methods for columns and for 3×3 blocks.

Because you've made a few minor changes to the class in the meantime, you transfer the entire code of the class in its current state to the prompt so that the AI tool has the correct starting point. (You can avoid this step if you use an AI wizard integrated in the editor or in the development environment.)

Prompt: I reorganized the code a bit. This is what the class looks like now. Please provide two more methods similar to `fillSingleEmptyInRow` to handle columns with only one cell missing and 3×3 areas with only one cell missing.

[code of the entire Sudoku class at this point in time, approximately 50 lines]

The code of the two new methods looks very similar to `fillSingleEmptyInRow`. You integrate the two methods into your code.

2.5.4 Test Code and Issues with the Test Data

To facilitate testing and debugging, ask ChatGPT to program two more methods. One should count the number of unsolved fields, while the other is supposed to display the sudoku in a legible form.

Prompt: Please provide code for two more methods for the `Sudoku` class: one is to count the empty cells, and the other one prints the sudoku with empty spaces instead of 0.

You integrate the largely trivial code of these methods into your project. We haven't printed the code here. If necessary, take a look at the sample files for the book.

The code previously supplied by ChatGPT didn't contain any errors. However, there are some issues with the test data. The sample sudoku generated by ChatGPT at the beginning of the session is too difficult for your program in its current configuration. So, you need to ask for simpler test data. You explicitly point out in the prompt that you only need the code for the array (once unsolved, once solved). Otherwise ChatGPT will output the entire code of the `Main` class again.

Prompt: I added calls to the two new methods to my `solve` method. However, the test puzzle you provided is too difficult at this time. Can you provide a slightly easier puzzle for testing? (Provide just the `int[][]` array and the solved puzzle for reference, no other code.)

The tests with the new sudoku are just as unsuccessful as with the original test data. You need a sudoku in which there is at least one row, one column, or one block in which only one number is missing. You formulate this information in another prompt.

Prompt: Even easier please. We need at least one row, column, and block with only one cell missing to test the code so far.

Absurdly, ChatGPT also fails at the third attempt to deliver a sufficiently simple sudoku puzzle. You can also take a completely solved sudoku and replace various cells with 0 in the sense of "not completed" and continue your tests with it.

2.5.5 Brainstorming

This brings you to the point where your program works in principle, but only for very simple sudoku puzzles. Now you don't really know what to

do. You ask ChatGPT for advice. For the AI system to get an idea of what your solution method currently looks like (you've written this code with calls to all other methods yourself), you want to pass this method to the prompt. Alternatively, you could add the file of the entire class to the prompt using drag and drop. In the prompt, you *do not* explicitly ask for new code, but for ideas on how you could further improve the program.

Prompt: I added a loop to my solve method. It calls all three `fillSingleXxx` methods until they are unable to solve a single cell.

```
// in the Sudoku class
public boolean solve() {
    int n;
    do {
        n = 0;
        n += fillSingleEmptyInRow();
        n += fillSingleEmptyInColumn();
        n += fillSingleEmptyInBlock();
        System.out.println("Filled " + n + " cells.");
    } while (n > 0);
    if (countEmptyCells()==0) {
        System.out.println("Sudoku puzzle solved!");
        return true;
    }
    System.out.println("Could not solve the puzzle.");
    printSudokuWithSpaces();
    return false;
}
```

So far, my program can solve only very simple sudokus. Can you suggest a more sophisticated solution method? Just present the idea, no code please.

ChatGPT now presents common solution concepts for sudokus (*naked pairs/triples*, *hidden pairs/triples*, *X-wing*, etc.) on the one hand and proposes a recursive backtracking algorithm on the other. First, let the AI tool explain the concept of *naked pairs/triples*.

Prompt: Let's start with naked pairs/triples. Can you show me a sudoku that can be solved with this concept so I can better understand the idea?

2.5.6 Storing Solution Candidates per Field

ChatGPT explains how the fact that only very few numbers are possible with a pair of free cells is exploited, thus further reducing the number of options. This gives you a new idea. Wouldn't it make sense to generally store for each cell which numbers are still permissible there? This information would speed up any further search for a solution.

You formulate this idea in the prompt. Your question only refers to a suitable data structure. Of course, you could simply ask ChatGPT for a suggestion. But you want to learn how to program on your own and are thinking about it yourself instead. The AI tool is more for verifying your ideas.

Prompt: I want to add an instance variable with 9×9 sets to store candidates for each cell. I want each set to be initialized with a set of all numbers from 1 to 9. Is

```
private Set<Integer>[ ][ ] candidates;
```

the correct data type? Later, I will add a method to remove all numbers already used within the same row, column, or 3×3 block.

ChatGPT confirms that the data type is a good choice and also provides the code for the initial initialization without being asked. Each set is initialized with all nine numbers. You implement the changes and then commission the next method.

In the prompt, you explain what the method should do: run through all 81 fields of the sudoku and eliminate from the set of possible solutions for this field all digits that already occur in this row, column, or block. This leaves only the solution candidates. Programming this method manually wouldn't be particularly difficult, but it would be tedious and prone to errors due to the many nested loops.

Prompt: Next, I need a method to loop over all the cells. For each cell, remove entries from the candidate set if the same number is used within the row, column, or 3×3 block. Can you provide code to do this?

You add the new `updateCandidates` method and call it after the three `fillSingleXxx` methods. In other words: First, the solution algorithm tries to find rows, columns, or 3×3 blocks with only one free cell. Second, `updateCandidates` eliminates all numbers already used in a row, column, or 3×3 block from the set of solution candidates. If there is now a cell for which there is only *one* possible solution, another field of the sudoku has been solved.

2.5.7 Filling in Fields Where Only One Solution Number Is Possible

In the next prompt, ask your AI junior programmer to program another method that searches for exactly such fields and enters the solution number there. As before, the method should count how many fields are solved in this way.

Prompt: Write a method to loop over all the cells that are still empty. If the candidate set contains exactly one number, set that cell accordingly. Again, the method should return the number of cells solved.

You integrate the new method into the loop for solving sudoku puzzles, and lo and behold, the program can now solve even moderately difficult puzzles. At this point, you're satisfied with your work and the AI contributions, so you close the project for the time being. The `Sudoku` class now has almost 300 lines. (You'll find the complete code in the sample files for the book.)

2.5.8 Personal Conclusion

I programmed a sudoku solver in the Kotlin language a few years ago without the help of AI. It took me about two days to get to the state of the program outlined here. I enjoyed programming back then, especially researching different sudoku solving strategies. I then added one or two more methods and finally implemented a recursive search algorithm. The time required for this was approximately another two days (you can read more about this at <https://kofler.info/sudokus-mit-kotlin-loesen>, although you'll need to translate from German to English).

So, when I started this example, I already had some prior knowledge. Nevertheless, it was amazing that after *two hours* with the help of AI, I was

about as far as I was on my first attempt after *two days*!

What I like best about this example is that even with this new implementation, I have the feeling that this is *my* code. Sure, thanks to AI help I have saved myself the tedious formulation of the nested loops. This isn't difficult, but it's prone to errors. But the AI-generated code is clearly understandable to me, and I could have changed it at any point or corrected it if necessary. (The latter wasn't necessary. The code generated by ChatGPT-4o was error free.)

Why did it work so well? Well, first, because there are various sudoku solvers available on the web, there was sufficient material at hand to train the language model. And secondly, because my own knowledge was relatively extensive, I set the direction of the code development, and I was able to ensure that the code made sense and that it was correct for each new method—almost at a glance.

This is exactly where I see the ideal conditions for the use of AI in coding: you know what you're doing and aren't overwhelmed by the complexity of the AI-generated code. You can easily assess whether the code works.

Admittedly, these prerequisites don't apply if you're just learning to program and solving typical exercises is indeed a major challenge. Ideally, you can also learn from the AI suggestions. However, you'll need a lot of self-discipline: your coding knowledge will only grow if you solve subproblems yourself first and question the code provided by the AI tool until you've understood it in detail.

2.5.9 Alternative Procedure

In this example, we've specified relatively clearly which strategies should be used to solve the sudoku. We've chosen a very "human" approach and used code to implement how we would solve a sudoku ourselves.

As the following prompt shows, one alternative is to leave the solution path entirely to the AI tool. However, this reduces your own contribution to the code to nothing.

Prompt: Can you show me a short, concise way to solve a sudoku with Java?

Most AI tools then suggest the backtracking algorithm. This is how it works in principle: The numbers 1, 2, and 3 to 9 are inserted into each free cell in turn. As soon as a number doesn't cause any obvious errors (no duplicates in the row, column, or 3×3 block), the method continues this procedure for the next free cell. If none of the numbers 1 to 9 fits there, a mistake was obviously made earlier. At this point, the method then terminates, goes back to the previous field, and inserts the next number there.

The backtracking algorithm is recursive, that is, it's based on the fact that a method can call itself repeatedly. The recursive approach has the advantage that it's easy to return to an earlier stage of the solution search in the event of errors. This is where the name of this concept comes from.

However, this approach is completely “unhuman”. Instead of starting with fields where there are obvious, simple solutions, the program systematically tries out all possibilities. The recursive approach is also very difficult to understand, especially if you're just learning to program. You can ask the AI tool to explain the code to you.



you add some comments to the code and explain the relevant concepts?

Although the code with the backtracking algorithm is difficult to understand, it has two decisive advantages over the solution presented earlier:

- The code can be implemented in just a few lines. The entire `SudokuSolver` class (see the sample files) including `main` and a test sudoku takes up only 70 lines.
- The algorithm can cope with sudokus of any difficulty.

2.5.10 Prompting Correctly

This book isn't about solving programming issues, regardless of whether they are sudokus, REST APIs, or database applications. It's rather about learning to use AI tools efficiently and responsibly.

In this section, we've shown you how to solve a complex problem using the support of AI. As an alternative approach, however, we've also demonstrated that you may reach your goal much faster if you use the capabilities of AI tools without restraint. (This doesn't always work as well as when solving sudoku!)

We believe that the second way is the wrong one:

- If you're still learning, the first way will give you a basic understanding of how to work with arrays, structure the code, and so on. Even though many methods come from the AI tool, you can still add code yourself and learn a lot. This isn't the case with the alternative approach.
- Even if you're only interested in solving an issue quickly, AI-generated code that is incomprehensible to you isn't desirable. Ultimately, you're responsible for your code and must be able to explain, adapt, or correct it if necessary. No matter what level of IT knowledge you currently have, you should always use AI tools within the scope of your knowledge, not beyond it.